WEEK-03

PRIMS AND KRUSHKAL'S ALGORITHM

Start here | *prims.c

```c
#include <stdio.h>
#include <limits.h>
#include <time.h>
#define MAX 100

/* ----------------- MATRIX METHOD -----------------
   Time Complexity: O(V^2)
------------------------------------------------- */

int minKey(int key[], int visited[], int n) {
    int min = INT_MAX, index = -1;

    for (int v = 0; v < n; v++) {
        if (!visited[v] && key[v] < min) {
            min = key[v];
            index = v;
        }
    }
    return index;
}

void primMatrix(int graph[MAX][MAX], int n) {
    int parent[MAX], key[MAX], visited[MAX];

    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        visited[i] = 0;
    }

    key[0] = 0;
    parent[0] = -1;

    for (int count = 0; count < n - 1; count++) {
        int u = minKey(key, visited, n);
        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] && !visited[v] && graph[u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
```

```c
    int total = 0;
    printf("\nMST (Adjacency Matrix):\n");

    for (int i = 1; i < n; i++) {
        printf("%d - %d  weight = %d\n", parent[i], i, graph[i][parent[i]]);
        total += graph[i][parent[i]];
    }

    printf("Total weight = %d\n", total);
}

/* ----------------- HEAP METHOD -----------------
   Time Complexity: O(E log V)
------------------------------------------------- */

typedef struct {
    int vertex;
    int key;
} Node;

void swap(Node *a, Node *b) {
    Node temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(Node heap[], int n, int i) {
    int smallest = i;
    int l = 2*i + 1;
    int r = 2*i + 2;

    if (l < n && heap[l].key < heap[smallest].key)
        smallest = l;

    if (r < n && heap[r].key < heap[smallest].key)
        smallest = r;

    if (smallest != i) {
        swap(&heap[i], &heap[smallest]);
        heapify(heap, n, smallest);
    }
}
```

Start here | *prims.c

```c
int extractMin(Node heap[], int *n) {
    Node root = heap[0];
    heap[0] = heap[*n - 1];
    (*n)--;
    heapify(heap, *n, 0);
    return root.vertex;
}
void primHeap(int graph[MAX][MAX], int n) {
    int parent[MAX], key[MAX], inMST[MAX];
    for (int i = 0; i < n; i++) {
        key[i] = INT_MAX;
        inMST[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;

    Node heap[MAX];

    for (int i = 0; i < n; i++) {
        heap[i].vertex = i;
        heap[i].key = key[i];
    }
    int heapSize = n;
    while (heapSize > 0) {
        int u = extractMin(heap, &heapSize);
        inMST[u] = 1;

        for (int v = 0; v < n; v++) {
            if (graph[u][v] && !inMST[v] && graph[u][v] < key[v]) {
                key[v] = graph[u][v];
                parent[v] = u;

                for (int i = 0; i < heapSize; i++) {
                    if (heap[i].vertex == v) {
                        heap[i].key = key[v];
                        break;
                    }
                }
            }
        }
        heapify(heap, heapSize, 0);
    }

```

```c
    int total = 0;
    printf("\nMST (Heap):\n");

    for (int i = 1; i < n; i++) {
        printf("%d - %d  weight = %d\n", parent[i], i, graph[i][parent[i]]);
        total += graph[i][parent[i]];
    }
   printf("Total weight = %d\n", total);
}

/* ---------------- MAIN ---------------- */

int main() {
    int n, graph[MAX][MAX];
    int choice;
    clock_t start, end;
    double time_taken;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix (weights):\n");
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &graph[i][j]);

    printf("\nChoose Method:\n");
    printf("1. Adjacency Matrix (O(V^2))\n");
    printf("2. Priority Queue / Heap (O(E log V))\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    start = clock();

    if (choice == 1) {
        primMatrix(graph, n);
    } else if (choice == 2) {
        primHeap(graph, n);
    } else {
        printf("Invalid choice!\n");
        return 0;
    }

    end = clock();

    time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Time taken = %f seconds\n", time_taken);

    return 0;
}
```

```
C:\Users\BMSCECSE\Docume

Enter number of vertices: 4
Enter adjacency matrix (weights):
0 2 3 4
2 0 7 4
3 7 0 5
4 4 5 0

Choose Method:
1. Adjacency Matrix (O(V^2))
2. Priority Queue / Heap (O(E log V))
Enter choice: 1

MST (Adjacency Matrix):
0 - 1  weight = 2
0 - 2  weight = 3
0 - 3  weight = 4
Total weight = 9
Time taken = 0.001000 seconds

Process returned 0 (0x0)   execution time : 10.206 s
Press any key to continue.
```

Start here | KRUSHKAL.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX 100

// Structure for edge
struct Edge {
    int u, v, weight;
};

int parent[MAX];

// Find (with path compression)
int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

// Union
void unionSet(int a, int b) {
    parent[a] = b;
}

// Sort edges (Bubble Sort)
void sortEdges(struct Edge edges[], int E) {
    struct Edge temp;
    for (int i = 0; i < E - 1; i++) {
        for (int j = 0; j < E - i - 1; j++) {
            if (edges[j].weight > edges[j + 1].weight) {
                temp = edges[j];
                edges[j] = edges[j + 1];
                edges[j + 1] = temp;
            }
        }
    }
}

```

Start here | KRUSHKAL.c

```c
int main() {
    int V, E;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");
    scanf("%d", &E);

    struct Edge edges[E];

    printf("Enter edges (u v weight):\n");
    for (int i = 0; i < E; i++) {
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].weight);
    }

    // MakeSet
    for (int i = 0; i < V; i++)
        parent[i] = i;

    // Start time
    clock_t start = clock();

    // Sort edges
    sortEdges(edges, E);

    printf("\nEdges in MST:\n");

    int totalCost = 0;
    int count = 0;

    for (int i = 0; i < E && count < V - 1; i++) {

        int u = find(edges[i].u);
        int v = find(edges[i].v);

        // If no cycle
        if (u != v) {
            printf("%d - %d  weight = %d\n",
                   edges[i].u, edges[i].v, edges[i].weight);

            totalCost += edges[i].weight;

```

```c
            unionSet(u, v);
            count++;
        }
    }

    printf("Total cost of MST = %d\n", totalCost);

    // End time
    clock_t end = clock();

    double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("Time taken = %f seconds\n", time_taken);

    /*
    Time Complexity:
    Sorting edges → O(E log E)
    Union-Find → O(E log V)
    Overall → O(E log E)
    */

    return 0;
}
```

```
C:\Users\BMSCECSE\Docume

Enter number of vertices: 5
Enter number of edges: 7
Enter edges (u v weight):
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
2 4 7
3 4 9

Edges in MST:
0 - 1  weight = 2
1 - 2  weight = 3
1 - 4  weight = 5
0 - 3  weight = 6
Total cost of MST = 16
Time taken = 0.002000 seconds

Process returned 0 (0x0)   execution time : 10.790 s
Press any key to continue.
```